

# Spec Driven Development - Design Document

---

**Date:** 2026-03-24 **Status:** Under Review **Purpose:** Establish a Spec Driven Development (SDD) framework for cross-functional product teams

## Executive Summary

This document defines a Spec Driven Development framework that bridges the gap between Product Management's business planning (Epics in ProductBoard/Jira) and Engineering's implementation needs (detailed Spec files in Git). The framework addresses common challenges: incomplete requirements, unclear handoffs between PM/Design/Engineering, information scattered across multiple tools, and knowledge loss during team transitions.

**Key Innovation:** A hybrid approach where Epics remain the business planning artifact (WHAT & WHY) and Spec files become the implementation blueprint (HOW), creating clear separation of concerns while maintaining a single source of truth for each domain.

## Problem Statement

The team currently uses multiple tools for product development:

- **ProductBoard:** Roadmaps, Program Epics, Epics, Design Epics
- **Jira:** Epics, Stories, Spikes (linked to design files)
- **Confluence:** Discovery and requirements documentation
- **Mural:** Service Blueprints, Journey Maps, Story Maps
- **Dovetail:** User research, business process research, market research
- **Figma:** Design files with context and annotations

### Current Challenges:

1. Requirements incomplete or change frequently during development
2. Handoffs between PM/Design/Engineering are messy or incomplete
3. Engineers start building before all details are thought through
4. Hard to validate that what was built matches what was intended
5. Information overload or poor readability of existing documentation
6. Knowledge loss when team members transition
7. AI coding assistants lack structured context for implementation

**Ideal State:** PM and Product Design conduct discovery → PD provides user/process/data insights → PM defines business value → Engineering collaborates on technical approach → Team builds from shared understanding.

**Reality:** Process consistency varies widely depending on feature complexity and team members involved.

## Design Goals

1. **Consolidation:** Single implementation blueprint that references (not duplicates) information from existing tools

2. **Completeness:** Template ensures nothing gets missed across PM/Design/Engineering disciplines
3. **Alignment:** All roles review and approve before implementation begins
4. **AI-Friendly:** Spec files in Git, version-controlled with code, accessible to AI coding assistants
5. **Process Adoption:** Framework augments (not replaces) existing Epic workflow to minimize disruption
6. **Future-Proofing:** Documentation serves current team and future team members

## Solution: Hybrid Epic + Spec Framework

The Boundary: Epic vs Spec

### Epic (ProductBoard/Jira) - The Business Case

- **Owner:** Product Manager
- **Purpose:** Business planning, prioritization, delivery tracking
- **Contains:**
  - User problem and business opportunity
  - Success metrics (KPIs, OKRs)
  - Links to research (Dovetail), journey maps (Mural), service blueprints
  - High-level scope and acceptance criteria ("Users can export their data")
  - Team assignments, dependencies, timeline
  - Business value and priority

### Spec File (Git Repo) - The Implementation Blueprint

- **Owner:** Collaborative (PM drafts context, Design adds UI/UX, Engineering adds technical approach)
- **Purpose:** Align team on HOW to build; provide AI-friendly implementation context
- **Contains:**
  - Link back to Epic for business context
  - Detailed user flows and design decisions (consolidates from Figma)
  - Technical architecture and component breakdown
  - Data models and API contracts
  - Detailed acceptance criteria ("Export button top-right, supports CSV/JSON, max 10k rows...")
  - Edge cases and error handling
  - Testing strategy

### The Relationship:

- **One Epic → One Spec:** Common case for straightforward features
- **One Epic → Multiple Specs:** Complex work broken into phases (backend, frontend, personalization)
- **Multiple Epics → One Spec:** Rare; shared technical foundation supporting multiple business features

### The Handoff Flow:

```
Epic created (PM)
  → Discovery & Research
  → Spec drafted (PM/Design/Eng collaborate)
  → Spec reviewed & approved
```

- Implementation begins
- Epic tracks delivery

## Spec Template Structure

**Location:** `specs/YYYY-MM-DD-feature-name.md` (version-controlled in Git)

### Sections:

#### 1. Context & Background

- **Owner:** Product Manager
- Links to Epic, research insights, user journey context, problem summary
- *Note:* Information sourced directly from Epic (link and summarize, don't duplicate)

#### 2. User Experience

- **Owner:** Product Designer
- **Contributors:** PM (user perspective), Engineering (technical constraints)
- User flows, UI/UX decisions, Figma links with written rationale, interaction patterns, accessibility, edge cases

#### 3. Technical Approach

- **Owner:** Engineering Lead
- **Contributors:** Design (constraints), PM (scope)
- Architecture, component breakdown, data models, API contracts, integrations, technology choices, performance, security

#### 4. Acceptance Criteria

- **Owner:** Shared (PM drafts, Design & Eng refine)
- Detailed testable criteria, edge cases, error states, performance requirements, accessibility, browser/device support

#### 5. Testing Strategy

- **Owner:** Engineering
- **Contributors:** PM (user scenarios), Design (visual/interaction QA)
- Unit test coverage, integration scenarios, manual QA focus, UAT approach, performance testing

#### 6. Implementation Plan (Optional)

- **Owner:** Engineering
- Phasing strategy, dependencies, effort estimation, feature flags, rollout strategy

#### 7. Open Questions

- **Owner:** Shared
- Unresolved decisions, assumptions needing validation, risks/mitigation, blockers to approval

## 8. References

- **Owner:** Shared
- Links to Figma, Mural, Dovetail, Confluence, related specs, technical docs

### Ownership Model:

- **Owner** = Responsible for drafting and maintaining completeness
- **Contributors** = Provide input, review, and refine
- **Shared** = All roles participate equally

Workflow: Epic to Spec to Implementation

### Step 1: Epic Creation (PM)

- PM creates Epic in ProductBoard/Jira with business context
- Epic contains: problem, value, success metrics, research links, high-level scope
- Epic status: "Discovery" or "Ready for Spec"

### Step 2: Spec Initiation (PM)

- PM creates spec file: `specs/YYYY-MM-DD-feature-name.md`
- PM fills "Context & Background" section
  - *Note: This information should match or be sourced directly from the Epic in ProductBoard/Jira - link to the Epic and summarize/reference key points rather than rewriting*
- PM drafts initial "Acceptance Criteria"
- PM commits to repo, opens PR or shares link with team
- Spec status: "Draft"

### Step 3: Design Contribution (Designer)

- Designer fills "User Experience" section
- Designer reviews and refines Acceptance Criteria (adds accessibility, interaction details)
- Designer adds any Open Questions about user flows or edge cases
- Spec status: Still "Draft"

### Step 4: Engineering Contribution (Engineering Lead)

- Engineering fills "Technical Approach" and "Testing Strategy"
- Engineering reviews entire spec for feasibility
- Engineering refines Acceptance Criteria (adds technical details)
- Engineering adds Open Questions about dependencies, risks, effort
- Spec status: Still "Draft"

### Step 5: Collaborative Review (Full Team)

- Team meeting or async review to resolve Open Questions
- Discuss trade-offs, scope adjustments, phasing decisions
- All three roles sign off (comment on PR or add approval in spec)
- Spec status: "In Review" → "Approved"

### Step 6: Implementation Begins (Engineering)

- Epic status in Jira: "In Progress"
- Engineering works from Spec as blueprint
- Spec is the reference for implementation details
- AI coding assistants can reference spec file for context

### Step 7: Scope Changes During Implementation

- Small changes: update spec inline, note in commit message
- Significant changes: team discussion, update spec, re-review if needed
- Keep spec and implementation in sync

### Step 8: Completion

- Epic status: "Done" (in Jira)
- Spec status: "Implemented"
- Spec remains as historical record for future reference

### Key Principles:

- No coding starts until Spec status = "Approved"
- All three roles (PM/Design/Eng) must contribute before approval
- Spec lives in Git, Epic lives in ProductBoard/Jira
- Spec is versioned with code, Epic tracks delivery status

### Granularity & Scoping

**Guiding Principle:** A spec should be sized for a coherent unit of work that can be understood, reviewed, and implemented as a whole.

### One Epic → One Spec (Common Case)

- Feature is relatively self-contained
- Can be implemented in 1-3 sprints
- Single user journey or workflow
- Example: "Add CSV export functionality to reports"

### One Epic → Multiple Specs (Complex Work)

- Epic represents a large initiative spanning multiple releases
- Natural phases or incremental value delivery
- Each spec can be approved and implemented independently
- Example Epic: "Revamp user dashboard"
  - Spec 1: `2026-03-dashboard-data-layer.md` (backend/API changes)
  - Spec 2: `2026-04-dashboard-ui-components.md` (frontend components)
  - Spec 3: `2026-05-dashboard-personalization.md` (user customization)

### Multiple Epics → One Spec (Rare)

- Shared technical foundation supporting multiple business features
- Example: Multiple Epics need a common authentication service
  - Spec: `2026-03-oauth-service.md` references all dependent Epics

When NOT to Create a Spec

- Bug fixes (unless architecturally significant)
- Minor copy changes or CSS tweaks
- Configuration changes
- Hotfixes or urgent patches

Questions to Ask When Sizing:

- Can this spec be reviewed in one sitting (20-30 min read)?
- Does it represent a complete user-facing feature or technical capability?
- Can it be implemented without waiting for other specs to complete?

**Pilot Guidance:** Start with medium-complexity feature, One Epic → One Spec relationship, estimated 1-2 sprint effort. Partner with PM and Engineering to refine granularity based on pilot learnings.

Maintaining Epic-Spec Sync

Source of Truth Boundaries:

| Aspect                            | Source of Truth                 | Other Location                   |
|-----------------------------------|---------------------------------|----------------------------------|
| Business value, success metrics   | <b>Epic</b> (ProductBoard/Jira) | Spec references/links to Epic    |
| User problem, research links      | <b>Epic</b> (ProductBoard/Jira) | Spec summarizes with link back   |
| High-level scope                  | <b>Epic</b> (ProductBoard/Jira) | Spec provides detailed breakdown |
| Detailed user flows, UI decisions | <b>Spec</b> (Git)               | Epic links to Spec               |
| Technical architecture            | <b>Spec</b> (Git)               | Epic may note dependencies       |
| Detailed acceptance criteria      | <b>Spec</b> (Git)               | Epic has high-level criteria     |
| Team assignments, timeline        | <b>Epic</b> (ProductBoard/Jira) | Spec notes team at creation      |
| Status tracking                   | <b>Epic</b> (ProductBoard/Jira) | Spec notes implementation status |

When Scope Changes:

Small Changes (during implementation):

- Update Spec file inline
- Note change in commit message
- Optional: add comment to Epic noting spec was updated

Significant Changes (affects business value or timeline):

- Update Epic first (impacts roadmap/reporting)
- Team discussion to align on change
- Update Spec to reflect new direction
- May require re-review/re-approval of Spec

Post-Implementation:

- Epic status → "Done" (Jira)
- Spec status → "Implemented"
- Both remain as historical record
- Spec provides technical context for future changes
- Epic provides business context and metrics

### Preventing Drift:

- Include Epic link at top of every Spec (easy to cross-reference)
- During Spec review, verify alignment with Epic scope
- If Epic scope changes significantly, flag for Spec update
- Treat Spec as "implementation snapshot" - captures detailed decisions at time of build

## Pilot Execution Plan

### Pilot Goals:

1. **Primary:** Team adopts the Spec process and finds it valuable enough to continue
2. **Secondary:** Reduce back-and-forth, improve alignment, create better reference materials

### Before the Pilot Feature:

1. Create the Spec template in repo ([specs/TEMPLATE.md](#))
2. Create [specs/README.md](#) explaining the Epic/Spec framework
3. Brief the pilot team (PM, Designer, Engineer) on the workflow
4. Set expectations: this is an experiment, we'll retrospect and adjust

### During the Pilot Feature:

1. PM creates Epic in ProductBoard/Jira as usual
2. PM initiates Spec using template
3. Designer and Engineering contribute their sections
4. Team reviews and approves together before coding
5. Engineering references Spec during implementation
6. Track: How many times did team reference the Spec? How many Slack questions were avoided?

### After the Pilot Feature:

1. Retrospective with pilot team:
  - What worked well?
  - What felt like overhead or busywork?
  - What was missing from the template?
  - Would you use this process again?
2. Gather specific feedback:
  - Was the Epic/Spec boundary clear?
  - Was ownership of sections clear?
  - Did the Spec reduce confusion during implementation?
  - Did AI assistants benefit from having the Spec?
3. Refine the template and workflow based on feedback
4. Decide: expand to more teams, adjust and re-pilot, or shelve

**Success Metrics** (observe during pilot):

- Spec completion rate (did all sections get filled?)
- Time to approval (was the review process smooth?)
- Implementation confidence (did Engineering feel clear on what to build?)
- Reference frequency (did team actually use the Spec during dev?)
- Rework reduction (fewer "wait, I thought we were doing X" moments)

**Recommendations for Pilot Feature Selection:** Work with PM and Engineering partners to choose a feature that:

- Is medium complexity (not too simple, not overwhelming)
- Involves all three roles (PM, Design, Engineering)
- Has clear design and technical components
- Is ~1-2 sprints in duration
- Isn't on a critical deadline (gives room to experiment)

## Evolution & Continuous Improvement

**The template and workflow should evolve based on team feedback.** Spec Driven Development is a practice, not a rigid process.

After the pilot, you may discover:

- Some sections are always empty (remove from template)
- New sections are needed (add to template)
- Ownership model needs adjustment (update guidelines)
- Granularity needs tuning (update "when to create" guidance)

**Iterate based on real usage patterns, not theoretical perfection.**

## Implementation Artifacts

This design will produce three key artifacts:

1. **This design document** (<docs/superpowers/specs/2026-03-24-spec-driven-development-design.md>)
  - Comprehensive reference capturing all decisions and rationale
2. **Spec README** (<specs/README.md>)
  - Quick-start guide for team: when to create specs, Epic/Spec relationship, workflow
3. **Spec Template** (<specs/TEMPLATE.md>)
  - Starting point for new specs with all sections, ownership, and instructions

## Conclusion

This Spec Driven Development framework provides a structured yet flexible approach to product development that:



- Respects existing PM/business processes in ProductBoard/Jira
- Gives Engineering a detailed implementation blueprint in Git
- Creates clear separation of concerns (WHAT/WHY vs HOW)
- Consolidates scattered information without duplicating work
- Enables AI coding assistants to reference structured context
- Serves both current and future team members

**Next Steps:** Create implementation artifacts and execute pilot with selected feature.

---

## Navigation

**Team Docs:** [Team README](#) · [Spec Template](#) · [Example Spec](#) · [Status Tracking](#)

**Guides:** [AI Integration](#) · [Pilot Discussion](#)

**Framework:** [Design Document](#) · [Implementation Plan](#) · [CLAUDE.md](#)